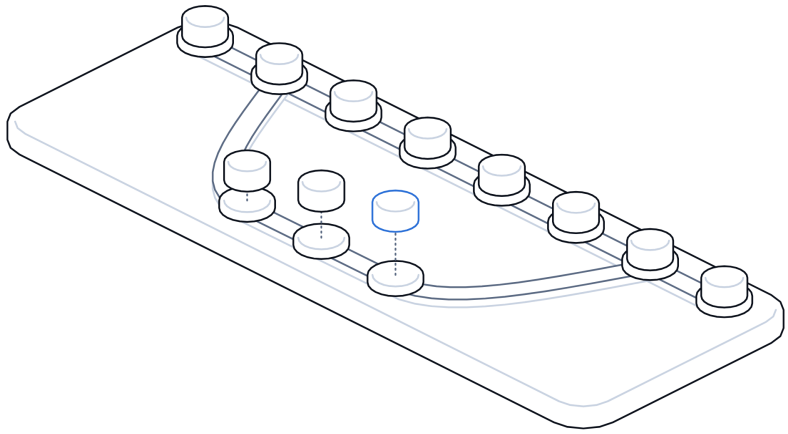


INTELLIGENCE REPORT · AI TOOLING SUPPLY CHAIN

Poisoned tools.

How attackers turned an AI gateway, a coding assistant extension and an AI SDK against the developers who trusted them in October 2026, and what it means for every organisation building with AI.

INSIDE

Contents

An evidence-based account of October 2026's attacks on AI developer tooling: what happened, how the attacks worked, why AI tools are such an attractive target, and what UK organisations should do now.

Executive summary 03

Key findings 04

At a glance 05

Timeline 06

01 Anatomy of the attacks 07

02 Why AI tooling is a target 13

03 What it means for UK organisations 17

Sources 22

EXECUTIVE SUMMARY

October 2026 was the month the AI developer toolchain became a mainstream target.

On 21 October a malicious release of LiteLLM, an open-source gateway that routes requests to many AI model providers, was published to PyPI and LiteLLM's Docker image tags for about two hours[1]. It carried a credential stealer that ran whenever Python started, and it targeted cloud keys, model provider keys and AI tool sessions[2]. LiteLLM traced the breach to an administrative key left in public commit history since May[3].

On 29 October two tampered versions of the Claude Code extension were served through Cursor's extension marketplace for roughly 11 hours, planting an attacker's SSH key and persistence on Windows and macOS machines[4]. Cursor says the versions were not developed by Anthropic, and Microsoft confirmed the official Visual Studio Marketplace and npm channels were not affected[4,5]. Earlier in the month a hijacked Tensorlake npm release carried a self-spreading worm that harvested AI tool and MCP configuration files[6].

The common thread is not a single flaw. Each attack abused a trusted release channel, and each went after the credentials that AI work concentrates on developer laptops, build runners and gateways. Signatures and provenance did not stop them, because the attackers used the projects' own pipelines or listings.

THE BOTTOM LINE

Treat AI gateways, coding assistants, extensions and MCP servers as privileged software. Know where they run, control how they are installed, keep keys scoped and short-lived, and be ready to rotate everything they could reach.

KEY FINDINGS

Seven things the October incidents tell us

01 **Trusted channels were the route in, not software bugs.**

The LiteLLM release was built by its own release workflow pointed at a lookalike fork, and the extension appeared under the Claude Code listing in Cursor's marketplace[3,4].

02 **Leaked secrets started the LiteLLM chain.**

An admin key committed in May was deleted within hours but never revoked, and it led to the GitHub tokens used in October[3].

03 **The payloads were built for AI environments.**

Researchers describe stealers that looked for model provider keys, AI tool sessions and MCP configurations alongside cloud and source control credentials[2,6].

04 **Speed of detection limited the damage, but not to zero.**

An external researcher flagged the LiteLLM release about 20 minutes after publication, yet stolen AWS credentials were being tested against Amazon Bedrock within hours, according to Wiz[3,7].

05 **Signatures and provenance passed.**

Compromised LiteLLM images carried a valid signature from the project's old key, and the Tensorlake release carried valid npm provenance[1,6].

06 **Uninstalling was not enough.**

Both the LiteLLM and extension payloads left startup files or persistence that survive removal of the package or extension[2,4].

07 **This is a pattern, not a one-off.**

LiteLLM was compromised for the second time in 2026, after a March incident linked to its CI security scanning tool[8].

AT A GLANCE

October in six numbers

Figures are from vendor advisories and named researchers. Some are estimates and are labelled as such.

~2 hours

How long the malicious LiteLLM 1.107.0 was live on PyPI on 21 October, from 11:39 to 13:23 UTC[1].

23 million

Approximate weekly downloads of LiteLLM, as estimated by Datadog Security Labs[2].

4+ months

How long the leaked admin key that began the LiteLLM chain stayed valid in public commit history[3].

~11 hours

How long the tampered Claude Code extension versions were available in Cursor's marketplace on 29 October[4].

5,000

The maximum number of users Cursor said may have installed the tampered extension, as reported by The Hacker News[9].

4.5 hours

Time after exfiltration before stolen AWS credentials were tested against Amazon Bedrock, according to Wiz[7].

TIMELINE

How October unfolded

All times UTC unless stated. Vendor timelines may still be revised.

27 May

A key is exposed

A configuration file holding an admin key for an internal LiteLLM platform is committed to a public repository and deleted three hours later, but stays in history[3].

8 Oct

Tensorlake npm release hijacked

tensorlake 0.5.144 is live on npm from 01:12 to 02:54 with a self-spreading credential stealer[6].

10 Oct

Tokens harvested

The attacker uses the key to obtain an organisation owner's GitHub CLI token, then exfiltrates CI secrets holding two more maintainers' tokens[3].

21 Oct, 11:39

Poisoned LiteLLM published

Seven malicious 1.107.0 wheels reach PyPI, followed by Docker Hub and GHCR images at 11:46[1].

21 Oct, 12:06

Detected and pulled

An external researcher reports the release. PyPI removes it by 13:23 and Docker tags are repointed by 14:20[3,1].

23 to 24 Oct

Root cause and new signing key

LiteLLM publishes the attack chain, rotates its image signing key and ships clean releases signed with the new key[3,1].

29 Oct

Tampered extension in Cursor

Claude Code extension 2.1.312 and 2.1.313 are served from about 09:27 to 20:35 UTC. Cursor removes them, shuts the marketplace, rotates keys and rebuilds[4,10].

30 Oct

Analysis and hardening

Cursor updates its indicators, Microsoft publishes an analysis and LiteLLM sets out its security changes[4,5,11].

01

PART 01

Anatomy of the attacks.

Three incidents, three routes in, one target: the credentials that AI work depends on.

01 LiteLLM: from leaked key to poisoned release

02 The extension and the worm

03 The incidents compared

LITELLM

From a forgotten key to a poisoned release

> **The key was deleted from the file within hours. It stayed valid for more than four months.**

LiteLLM is popular because it simplifies AI work: one gateway, many model providers, with central management of keys and spend. That role is also what made its compromise serious. Datadog estimates the package sees about 23 million downloads a week[2].

According to LiteLLM's root cause analysis, the chain began on 27 May 2026, when a configuration file containing an administrative key for an internal agent platform was committed to a public repository. The file was deleted about three hours later, but the commit stayed in public history and the key was never revoked[3].

On 10 October the attacker used that key against the platform's API. A request listing managed agents returned configuration that included the GitHub CLI token of one of two organisation owners. With it, the attacker ran temporary workflows that posted repository secrets, including two other maintainers' tokens, to an external endpoint, then deleted the runs[3].

On 21 October the attacker changed LiteLLM's private release workflow so it could build from a different source, overrode branch protection with the owner token and pointed it at a lookalike fork, Berri-AI/litellm. A second token approved the production gate, the poisoned wheels and images were published, and a third token deleted the run records[3]. LiteLLM also notes it had temporarily widened release permissions that week for bugfix backports[1].

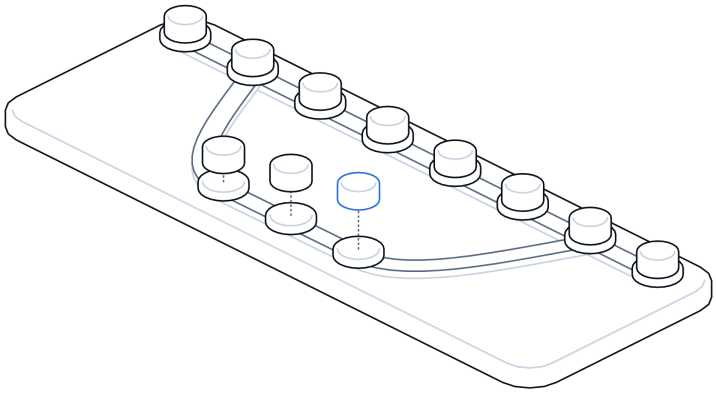
LiteLLM's initial notice also said it had seen Claude Code embed a user's GitHub CLI token in response bodies, which could then appear in gateway logs[1]. That is LiteLLM's observation; The Hacker News noted it had not been independently verified, and we found no public response from Anthropic on the point[12].

The payload ran from Python startup files, so it executed whenever Python started, even if LiteLLM was never imported. It encrypted what it found and sent it to a lookalike domain, and an hourly loader was ready to fetch further code, although only test values were seen[2]. Wiz researchers counted 16 data-harvesting modules[13]. Enterprise customers on LiteLLM's private release were not affected[1].

HOW IT WORKED

The LiteLLM attack chain in four moves

Each step used a legitimate credential or feature[3].



01 **A public commit leaks an admin key**

The file was deleted, but the key stayed in history and was never revoked[3].

02 **Tokens are collected**

The platform returned an owner's GitHub CLI token; rogue workflows leaked two more[3].

03 **The pipeline is pointed at a fork**

Protection was overridden and the release built from the lookalike Berri-AI/litellm[3].

04 **The payload hunts AI-era secrets**

Cloud keys, provider keys, AI tool sessions and LiteLLM's proxy database credentials[2].

EXTENSION AND WORM

An extension, a worm and a familiar playbook

> The extension did not need a vulnerability. It ran because extensions run.

The second major October incident hit developers' desktops. Between about 2:27am and 1:35pm Pacific time on 29 October, versions 2.1.312 and 2.1.313 of the Claude Code extension in Cursor's marketplace carried a malicious loader for Windows and macOS. Cursor states the versions were not developed by Anthropic[4].

Microsoft's analysis found the loader ran when the extension loaded, before the normal extension code. It gathered host details and the names, not values, of environment variables that looked like secrets, appended an attacker's SSH key, tried to enable SSH and open a reverse tunnel, and set up persistence disguised as system monitoring[5]. BleepingComputer reported that coding errors meant SSH was not enabled on most macOS and Windows hosts, although key planting and reconnaissance could still occur[14].

Cursor removed the versions, shut its entire marketplace as a precaution, audited access, rotated keys and rebuilt the marketplace before bringing it back[4]. Cursor told reporters at most 5,000 users were affected and that it would contact them directly[9]. Anthropic staff said the official VS Code and Open VSX channels were unaffected and that Anthropic's systems were not compromised[9]. How the attacker came to publish under the Claude Code listing has not been publicly explained[14].

Early reports that Open VSX also carried the malicious version are disputed. One security firm said it found it there, while others, including Microsoft's analysis, place the incident in Cursor's marketplace only[15,5].

The Tensorlake case on 8 October showed the same instinct in package form. A compromised maintainer account pushed code and published tensorlake 0.5.144 through the project's own workflow, so it carried valid npm provenance. Its install hook harvested tokens, cloud keys and AI tool and MCP configurations, then republished itself into other packages the victim could write to[6].

Tensorlake's payload also wrote Claude Code and VS Code settings files into repositories so it would run again when a project was opened[16], and its advisory warns that it installs a service that deletes the user's home directory if the stolen GitHub token is revoked before the service is removed[6].

HOW IT WORKED

Inside the tampered extension

The legitimate extension code was left in place, so the editor appeared to work normally while the loader ran first[14].



01 **Load**

The injected loader executed as soon as the editor loaded the extension, with no need to use Claude Code[5,14].

02 **Reconnoitre**

It collected hostname, user, Git email, public IP and up to 40 names of secret-looking environment variables[5].

03 **Plant access**

It appended an SSH key labelled svc-telemetry and tried to open a reverse SSH tunnel to the attacker's server[4].

04 **Persist**

A LaunchAgent on macOS or a scheduled task and Startup script on Windows kept retrying, so uninstalling the extension may not be enough[4].

SIDE BY SIDE

The three October incidents compared

Facts as reported by the vendors involved and named researchers.

	LITELLM 1.107.0	CLAUDE CODE EXTENSION	TENSORLAKE 0.5.144
Route in	Leaked key, stolen GitHub tokens, release workflow pointed at a fork[3].	Tampered versions in Cursor's marketplace; method not disclosed[4,14].	Compromised maintainer account and the project's own workflow[6].
Exposure	About two hours on PyPI, longer on Docker tags, 21 October[1].	About 11 hours on 29 October[4].	About 1 hour 40 minutes on 8 October[6].
Payload	Credential stealer plus hourly remote code loader[2].	SSH key, reverse tunnel and reconnaissance[5].	Credential stealer and self-spreading worm[6].
AI angle	Provider keys, AI tool sessions, gateway database[2].	Impersonated a leading AI coding assistant[4].	AI tool and MCP configs; wrote assistant settings into repos[6,16].
Survives removal	Yes: Python startup files[2].	Yes: startup persistence[4].	Yes: background service[6].
Vendor response	Pulled, keys rotated, new signing key, hardening plan[1,11].	Removed, marketplace rebuilt, keys rotated, server taken down[4].	Advisory, clean release path, rebuild guidance[6].

02

PART 02

Why AI tooling is a target.

AI work gathers keys, permissions and trust in exactly the places attackers can reach.

01 What attackers want from AI teams

02 A pattern years in the making

03 What helped, and what did not

THE ATTRACTION

Why the AI toolchain is worth attacking

> **Compromise an AI gateway and you inherit every key it holds.**

The October payloads read like shopping lists, and the lists are revealing. Alongside the usual cloud and source control credentials, researchers found the LiteLLM stealer looking for model provider API keys, sessions for Claude, Codex and Cursor, and the credentials LiteLLM's own proxy database holds for upstream providers[2]. Tensorlake's worm collected configuration and MCP files for several AI coding tools[16].

The reason is concentration. An AI gateway exists to hold many provider keys in one place. A coding assistant runs with a developer's own access to code, shell and tokens. An MCP server is, by design, a bridge from an AI assistant into other systems. Compromise any one of them and you inherit its reach.

Those keys have direct value. Wiz reported that the attacker behind the LiteLLM release validated stolen AWS credentials from CI runners and Kubernetes nodes and checked access to models on Amazon Bedrock, behaviour it linked to LLMjacking, where stolen access is used to run models at the victim's expense[7].

Pace adds risk. AI teams update gateways, SDKs and assistants often, to pick up new models and features, and many installs and container tags float to the newest version. A malicious release does not need to survive long if automated builds fetch it within minutes.

Trust adds more. Developers are used to approving extensions and packages from familiar names, and some protections that buyers rely on were not designed for this threat. LiteLLM's compromised images carried a valid signature from its old key, and Tensorlake's release had valid provenance, because both were produced by the projects' own release machinery[1,6].

Finally, AI agents themselves are being drafted in. Earlier attacks have tried to use locally installed AI coding tools to search machines for secrets, and Tensorlake's worm used assistant settings files to re-run itself[17,16]. The tools are both the target and, increasingly, the vehicle.

THE PATTERN

October did not come from nowhere

Earlier developer supply chain attacks rehearsed every technique seen in October.

DATE	INCIDENT	TECHNIQUE	ECHO IN OCTOBER
Aug 2025	Nx build system packages on npm[18]	Install script stole secrets; tried to use local AI coding CLIs with prompts disabled[17].	AI tools as both target and vehicle.
Sep 2025	Shai-Hulud npm worm[19]	Self-replicating worm across 500+ packages stealing tokens and cloud keys[19].	Tensorlake's Mini Shai-Hulud payload.
Feb 2026	Trivy extension on Open VSX[20]	Release designed to use a local AI coding agent to collect and exfiltrate data[20].	Poisoned editor extension.
Mar 2026	GlassWorm on Open VSX[21]	72+ malicious extensions, some mimicking AI coding assistants including Claude Code[21].	Impersonating trusted AI tools.
Mar 2026	LiteLLM 1.82.7 and 1.82.8[8]	Believed to stem from a compromised CI scanning dependency; credential stealer via a startup file[8].	Same project, same startup-file trick.

CONTROLS

What helped, and what did not

Controls that people rely on, measured against what actually happened in October.

CONTROL	WHAT HAPPENED IN OCTOBER	THE LESSON
Signing	Malicious LiteLLM images carried a valid signature from the old key[1].	Verify by digest against a known-good list, and rotate keys when tokens leak.
Provenance	Tensorlake's malicious release carried valid npm provenance[6].	Provenance proves where code was built, not that the account was legitimate.
Private registry	LiteLLM enterprise customers on the private release were not affected[1].	Controlled distribution and delayed promotion shrink exposure.
Branch protection	An owner token was used to override protection and force-push[3].	Owner rights should sit on separate accounts with no API or CLI use.
Community detection	An external researcher spotted the LiteLLM release within about 20 minutes[3].	A short cooling-off period lets others find problems first.

03

PART 03

What it means for the UK.

Most UK organisations are users of these tools, not builders. The exposure is real either way.

01 The UK picture

02 Recommendations

03 Questions for boards

UK ORGANISATIONS

What it means for UK organisations

> **The first question in every incident was the same: are we running it?**

Few UK businesses write their own AI gateways, but many now run one, often stood up quickly by a data or engineering team to give staff access to several models. Many more have developers using AI coding assistants and editor extensions on laptops that hold cloud and source control credentials. That makes these incidents relevant well beyond the technology sector.

The UK already has a framework for this. The NCSC's supply chain security guidance sets out 12 principles for gaining and keeping oversight of the suppliers your systems depend on[22]. The Software Security Code of Practice, published by DSIT and the NCSC in May 2025, sets voluntary baseline expectations of software vendors, including how they secure their build environments and communicate with customers[23]. Open-source projects and extension marketplaces sit awkwardly in procurement, but the principles transfer.

The most common gap we see is basic visibility. When an advisory lands, many organisations cannot say quickly which machines installed a given package version or extension, because developer tooling sits outside the asset and software inventories that IT maintains.

The second gap is how credentials are held. Model provider keys are often long-lived, broadly scoped, shared between projects and stored in environment files. That turns a two-hour exposure window into a long-tail problem, because every key has to be found and replaced.

The third is response. The October advisories were detailed and specific about order: do not start Python in an affected environment, remove persistence before revoking tokens, rotate from a clean machine. Teams that treated them as routine patch notices risked making things worse.

Data protection matters too. If a compromised machine or gateway held personal data or credentials to systems that do, organisations need to assess whether a personal data breach has occurred and whether it must be reported to the ICO within 72 hours under UK GDPR.

RECOMMENDATIONS

Six steps to keep AI tooling trustworthy

In rough priority order. Most need policy and configuration, not new products.

01 **Inventory AI tooling**

Gateways, SDKs, assistants, extensions and MCP servers, with an owner and what each can reach.

02 **Pin, delay and mirror**

Pin by version and digest, hold new releases for a cooling-off period, and install from a private registry you can block.

03 **Allow-list extensions and MCP servers**

Use central admin controls; Cursor, for example, documents publisher allow-lists and install cooldowns[24].

04 **Shrink the value of keys**

Per-application provider keys with spend limits, in a secrets manager, and short-lived tokens for source control and CI.

05 **Separate admin from daily use**

Owner and admin rights on dedicated accounts, as LiteLLM is now adopting[11].

06 **Watch behaviour and rehearse**

Alert on editors adding SSH keys or startup items, and practise cleaning and rotating in the order advisories prescribe.

FOR THE BOARD

Questions boards should ask

Short questions that reveal whether AI tooling risk is being managed.

EXPOSURE

- ? Do we know which AI gateways, coding assistants, extensions and MCP servers we run, and who owns each?
- ? If a named version were compromised today, how long would it take us to know whether we ran it?
- ? How many systems could a single stolen model provider key or developer token reach?

ASSURANCE

- ? Are new releases delayed and installed only from sources we control?
- ? When did we last rehearse rotating every key on a compromised developer machine?
- ? What do our key AI tooling suppliers commit to on release security and incident notice?

GLOSSARY

Terms used in this report

AI gateway

Software that sits between applications and AI model providers, holding provider keys and routing requests.

Supply chain attack

Compromising trusted software, such as a package or extension, to reach its users.

Release pipeline

The automated workflow that builds, signs and publishes a project's packages and images.

Lookalike repository

An account or repository named to resemble a legitimate one, such as Berri-AI for BerriAI.

Image digest

A fixed fingerprint for a container image. Tags such as latest can be moved; digests cannot.

Provenance

Signed evidence of where and how a release was built.

.pth file

A Python file processed when the interpreter starts, which can run code automatically.

Extension marketplace

A store from which code editors install add-ons, such as Open VSX or Cursor's marketplace.

MCP server

A connector using the Model Context Protocol that lets AI assistants use tools and data.

Reverse SSH tunnel

A connection from the victim to the attacker that lets the attacker reach the victim's machine.

LLMjacking

Using stolen credentials to run AI models at someone else's expense.

Worm

Malware that spreads itself, here by republishing into packages the victim can write to.

SOURCES

Sources and notes

Sources were accessed in November 2026. Where reports differ, we say so in the text. Early reporting on live incidents can change.

01 LiteLLM, Contained: Oct 21, 2026 LiteLLM release v1.107.0 Compromised (incident notice, updated 23 and 30 October), 21 October 2026, https://docs.litellm.ai/blog/security-incident-oct-21-2026

02 Datadog Security Labs, LiteLLM supply chain compromise: What you need to know, 21 October 2026, https://securitylabs.datadoghq.com/articles/litellm-compromised-pypi-docker/

03 LiteLLM, The attack chain behind the v1.107.0 LiteLLM release, 23 October 2026, https://docs.litellm.ai/blog/security-incident-oct-21-2026-attack-chain

04 Cursor, Security Notice: Malicious Anthropic.claude-code Versions in the Cursor Marketplace, 29 October 2026 (updated 30 October), https://cursor.com/blog/claude-code-extension-security-notice

05 Microsoft Security Blog, Inside a malicious Claude Code-branded extension and its persistent SSH backdoor, 30 October 2026, https://www.microsoft.com/en-us/security/blog/2026/10/30/inside-malicious-claude-code-branded-cursor-extension-persistent-ssh-backdoor/

06 Tensorlake, GitHub Security Advisory GHSA-8g63-53c9-65j2: Malicious tensorlake@0.5.144 published to npm, 8 October 2026, https://github.com/tensorlakeai/tensorlake/security/advisories/GHSA-8g63-53c9-65j2

07 Help Net Security, LiteLLM hit by second supply chain attack this year, users urged to rotate credentials, 22 October 2026, https://www.helpnetsecurity.com/2026/10/22/litellm-compromised-supply-chain-attack-pypi-docker/

08 LiteLLM, Security Update: Suspected Supply Chain Incident, 24 March 2026, https://docs.litellm.ai/blog/security-update-march-2026

09 The Hacker News, Official Claude Code Extension on Cursor Marketplace Trojanized with Persistent SSH Backdoor, 30 October 2026, https://thehackernews.com/2026/10/official-claude-code-vs-code-extension.html

10 Sophos, Malicious Claude Code extension distributed through Cursor marketplace, October 2026, https://www.sophos.com/en-us/blog/malicious-claude-code-extension-distributed-through-cursor-marketplace

11 LiteLLM, What we are changing after October 21, 30 October 2026, https://docs.litellm.ai/blog/security-changes-after-oct-21-2026

12 The Hacker News, LiteLLM Suspects GitHub Token Leak in AI Gateway Logs Behind Malicious Package Release, 22 October 2026, https://thehackernews.com/2026/10/litellm-suspects-github-token-leak-in.html

SOURCES

Sources and notes

Continued. Facts we could not verify against a primary or reputable source have been left out.

13 The Register, LiteLLM supply chain attack hoovers up passwords and secrets, 21 October 2026, https://www.theregister.com/a/5304548

14 BleepingComputer, Malicious Claude Code extension on Cursor planted SSH backdoors, 30 October 2026, https://www.bleepingcomputer.com/news/security/malicious-claude-code-extension-on-cursor-planted-ssh-backdoors/

15 Help Net Security, Claude Code extension poisoned with persistent SSH backdoor, 30 October 2026, https://www.helpnetsecurity.com/2026/10/30/claude-code-extension-ssh-backdoor/

16 The Hacker News, Tensorlake npm Package Compromised to Deliver Shai-Hulud Credential-Stealing Worm, 8 October 2026, https://thehackernews.com/2026/10/tensorlake-npm-package-compromised-to.html

17 StepSecurity, Supply chain security alert: popular Nx build system package compromised with data-stealing malware, August 2025, https://www.stepsecurity.io/blog/supply-chain-security-alert-popular-nx-build-system-package-compromised-with-data-stealing-malware

18 Nx (nrwl), GitHub Security Advisory GHSA-cxm3-wv7p-598c: Malicious versions of Nx and some supporting plugins were published, 27 August 2025, https://github.com/nrwl/nx/security/advisories/GHSA-cxm3-wv7p-598c

19 CISA, Widespread Supply Chain Compromise Impacting npm Ecosystem, 23 September 2025, https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem

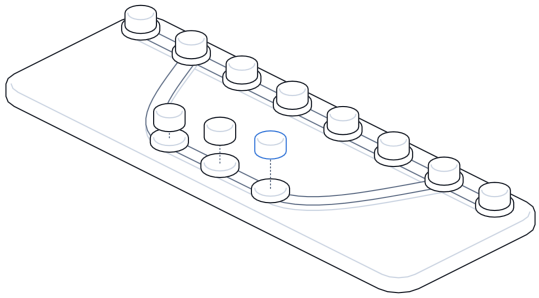
20 Aqua Security, GitHub Security Advisory GHSA-8mr6-gf9x-j8qg: Malicious Trivy OpenVSX Extension Release (CVE-2026-28353), 27 February 2026, https://github.com/aquasecurity/trivy-vscode-extension/security/advisories/GHSA-8mr6-gf9x-j8qg

21 The Hacker News, GlassWorm Supply-Chain Attack Abuses 72 Open VSX Extensions to Target Developers, 14 March 2026, https://thehackernews.com/2026/03/glassworm-supply-chain-attack-abuses-72.html

22 NCSC, Supply chain security guidance, accessed November 2026, https://www.ncsc.gov.uk/collection/supply-chain-security

23 DSIT and NCSC, Software Security Code of Practice, May 2025, https://assets.publishing.service.gov.uk/media/681a156fdf188ba858873aac/Software_Security_Code_of_Practice_Web_Accessible.pdf

24 Cursor, Security and privacy hardening (documentation), accessed November 2026, https://cursor.com/docs/enterprise/security-hardening



TRANSPUTEC

Poisoned tools need a practised response.

We help UK organisations inventory their AI tooling, harden how it is installed and respond quickly when a supplier is compromised.

Talk to our security team

transputec.com

PUT IT INTO PRACTICE

Companion playbook · Trust your tools